ATIVIDADES DA UNIDADE CURRICULAR

Unidade Curricular: Programação para Internet I

Semestre 3 – 60h

Sumário

1 Visão Geral da Unidade Curricular **3**

2 Roteiro Prático 01 — Fundamentos da Web, HTML5 e Versionamento **5**

2.1 Introdução 5
2.2 Parte 1 — Fundamentos da Internet 6
2.2.1 Como funciona uma página web? 6
2.3 Parte 2 — Estrutura Base do Documento HTML5 6
2.3.1 Estrutura mínima obrigatória 6
2.4 Parte 3 — Comandos Básicos HTML5 8
2.5 Parte 4 — Trabalhando com Links 10
2.5.1 Exemplo 10
2.6 Parte 5 — Versionamento com Git e GitHub 11
2.7 Checklist Final de Verificação 12

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional **13**

3.1 Introdução 13
3.2 Fundamentação Teórica — HTML5 Semântico 13
3.3 Principais Tags Estruturais do HTML5 14
3.4 Boas Práticas 14
3.5 Parte 1 — Organização Profissional do Projeto 16
3.6 Parte 2 — Estrutura HTML Semântica 16
3.7 Parte 3 — Introdução ao CSS Externo 18
3.8 Parte 4 — Inserindo Imagens 19
3.9 Checklist Final 20
3.10 Entrega da atividade 21
3.11 Fundamentação Complementar — Propriedades CSS Essenciais 22
3.12 Principais Propriedades CSS 22
3.13 Dicas Essenciais — Git e GitHub 23

4 Roteiro Prático 03 — Publicação com GitHub Pages e Aprimoramento Visual **24**
4.1 Parte 1 — Ajustes Estruturais 24
4.2 Parte 2 — Aprimoramento Visual com CSS 24
4.3 Parte 3 — Publicação com GitHub Pages 27
4.3.1 Passo 1 — Atualizar repositório 27
4.3.2 Passo 2 — Ativar GitHub Pages 27
4.3.3 Parte 4 — Verificação da Publicação 27

5 Checklist Final **28**

6 Roteiro Prático 04 — Evoluindo o Layout com Flexbox **29**
6.1 Objetivos da Aula 29
6.2 Introdução 29
6.3 Preparação do Ambiente — Recuperando o Projeto 29
6.3.1 Passo 1 — Clonar o Repositório Individual 29
6.3.2 Passo 2 — Acessar o Diretório 29
6.3.3 Passo 3 — Entrar na Pasta do Último Roteiro 30
6.4 Fundamentação Teórica — Flexbox 30
6.4.1 Ativando o Flexbox 30
6.4.2 Principais Propriedades 30
6.5 Visualização do Resultado Esperado 30
6.6 Parte 1 — Melhorando o Menu com Flexbox 31
6.7 Parte 2 — Organizando Conteúdo com Cards 32
6.7.1 Estrutura HTML 32
6.7.2 Estilização CSS 32
6.8 Parte 3 — Responsividade Básica 33
6.9 Checklist Final 34
6.10 Repositório Oficial da Unidade Curricular 34

1 Visão Geral da Unidade Curricular

Esta unidade curricular tem como objetivo desenvolver competências para a construção de sistemas para internet, integrando tecnologias frontend e server side, com utilização de banco de dados e aplicação prática em contexto real.

Ao longo do semestre, os estudantes irão projetar e implementar um sistema web funcional, aplicando conhecimentos de HTML, CSS, JavaScript, programação server side e integração com banco de dados.

A disciplina será orientada por uma perspectiva prática, incremental e profissional, incentivando organização, autonomia, raciocínio lógico e responsabilidade no desenvolvimento de soluções tecnológicas.

Estrutura do Semestre

O semestre está organizado em 6 etapas estruturantes, distribuídas ao longo dos encontros:

1. **Fundamentos da Web e Estrutura HTML**
   Compreensão da arquitetura cliente-servidor e construção de páginas estruturadas com HTML5.
2. **Estilização e Construção de Layouts**
   Aplicação de CSS, técnicas de layout moderno, responsividade e uso de framework CSS.
3. **Interatividade com JavaScript**
   Manipulação de DOM, eventos e validação de formulários.
4. **Programação Server Side**
   Desenvolvimento de aplicação web estruturada utilizando linguagem server side.
5. **Integração com Banco de Dados**
   Implementação de operações CRUD e persistência de dados em base relacional.
6. **Projeto Integrador com Extensão**
   Desenvolvimento e aplicação de sistema web para atender demanda real da comunidade.

Competências Desenvolvidas

Ao final da disciplina, o estudante deverá ser capaz de:

- Compreender o funcionamento da arquitetura web;
- Desenvolver interfaces estruturadas com HTML e CSS;
- Construir layouts responsivos utilizando framework CSS;
- Implementar interações básicas com JavaScript;

- Desenvolver aplicações web com programação server side;
- Integrar banco de dados em sistemas para internet;
- Projetar e implementar um sistema web funcional;
- Aplicar soluções tecnológicas em contexto comunitário.

Metodologia

A disciplina será conduzida por meio de aprendizagem baseada em projetos (Project-Based Learning), com desenvolvimento incremental de um sistema web ao longo do semestre.

Cada etapa envolverá momentos de fundamentação teórica, atividades práticas em laboratório e evolução progressiva do projeto.

A atividade de extensão será integrada ao projeto final, conduzindo os estudantes à aplicação da solução desenvolvida em contexto real.

Avaliação da Aprendizagem

A avaliação será processual, contínua e cumulativa, considerando a evolução técnica do projeto e o desempenho individual do estudante.

A nota final será composta por quatro componentes:

- **Etapa 1 – Frontend Estrutural (20%)**
  Implementação correta de HTML, organização semântica e estrutura adequada de navegação.
- **Etapa 2 – Layout e Interatividade (20%)**
  Aplicação de CSS, responsividade, uso de framework e implementação de interações com JavaScript.
- **Etapa 3 – Backend e Banco de Dados (30%)**
  Implementação funcional de rotas, formulários, persistência de dados e operações CRUD.
- **Etapa 4 – Projeto Integrador com Extensão (20%)**
  Aplicação do sistema desenvolvido para atender demanda real, incluindo documentação técnica e apresentação final.
- **Participação e Desempenho Individual (10%)**
  Considerando assiduidade, pontualidade, participação ativa nas aulas, organização, postura profissional e cumprimento de prazos.

A avaliação prioriza a coerência técnica do sistema desenvolvido, a integração adequada das tecnologias estudadas e a capacidade de aplicação prática dos conhecimentos adquiridos.

2 Roteiro Prático 01 — Fundamentos da Web, HTML5 e Versionamento

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Compreender a evolução da Internet e o papel do HTML;
- Identificar os principais componentes de um documento HTML5;
- Criar uma página web estruturada corretamente;
- Utilizar tags básicas para organização de conteúdo;
- Executar versionamento com Git e GitHub desde o início do projeto.

Introdução

A Internet transformou profundamente a forma como pessoas, empresas e governos produzem, compartilham e consomem informação.

Sua origem remonta à década de 1960, com a ARPANET, projeto militar norte-americano que buscava criar uma rede descentralizada de comunicação entre computadores.

Na década de 1990, Tim Berners-Lee desenvolveu a World Wide Web (WWW), criando:

- O protocolo HTTP;
- O conceito de URL;
- A linguagem HTML;
- O primeiro navegador web.

A Web evoluiu em diferentes fases:

- **Web 1.0**: páginas estáticas;
- **Web 2.0**: interação e redes sociais;
- **Web 3.0**: dados inteligentes e descentralização.

O HTML (HyperText Markup Language) é a base estrutural de praticamente toda aplicação web.

Mesmo sistemas complexos desenvolvidos em outras linguagens entregam HTML ao navegador.

Parte 1 — Fundamentos da Internet

2.2.1 Como funciona uma página web?

Quando um usuário digita uma URL no navegador:

1. O navegador envia uma requisição HTTP;
2. O servidor responde enviando um documento HTML;
3. O navegador interpreta o HTML;
4. O conteúdo é renderizado na tela.

Atividade Prática 1 - Pesquisa dirigida

Pesquise e escreva uma definição objetiva (máx. 4 linhas cada) para:

- ARPANET
- Tim Berners-Lee
- W3C
- HTTP
- URL
- Navegador Mosaic
- Web 1.0, 2.0 e 3.0

Em seguida, responda:
Qual foi o principal impacto da Web na transformação da sociedade digital?

Parte 2 — Estrutura Base do Documento HTML5

2.3.1 Estrutura mínima obrigatória

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Prog. Internet I - 2026</title>
</head>
```

```
<body>

</body>
</html>
```

Atividade prática 2 — Criando o Projeto

- Criar diretório `prog-internet-2026`;
- Criar arquivo `index.html`;
- Inserir a estrutura base HTML5;
- Testar no navegador.

Micro-Commit 1 — Estrutura Inicial

```
git init
git add .
git commit -m "Estrutura inicial HTML5"
```

Parte 3 — Comandos Básicos HTML5

TAG	Definição
<!DOCTYPE html>	Define documento HTML5
<html>	Estrutura geral da página
<head>	Informações da página
<meta charset="UTF-8»	Codificação
<title>	Título da aba
<body>	Conteúdo visível
<h1> até <h6>	Títulos
<p>	Parágrafo
<hr>	Linha horizontal
<ul> / <li>	Lista não ordenada
<pre>	Texto pré-formatado
<a>	Hiperlink

Atividade 3 — Inserindo Conteúdo

Adicionar ao `<body>`:

- Título da disciplina;
- Subtítulo com seu nome;
- Parágrafo explicando o que você entende por Internet;
- Lista com 3 tecnologias que deseja aprender;
- Duas linhas horizontais (`<hr>`);
- Bloco `<pre>` contendo:
  - Seu curso;
  - Seu semestre;
  - Data atual.

Após implementar, teste novamente no navegador.

Micro-Commit 2 — Conteúdo Estruturado

Registre a evolução do projeto:

```
git add .
git commit -m "Adicionando conteúdo estruturado"
```

Parte 4 — Trabalhando com Links

2.5.1 Exemplo

```
<a href="https://www.ifsc.edu.br" target="_blank">
    Página do IFSC
</a>
```

Atividade 4 — Trabalhando com Links e Comentários

Adicionar ao final da página:

- Um link para o site do IFSC;
- Um link para o site do W3C;
- Um comentário HTML explicando qual foi sua maior dificuldade na atividade.

Os links devem abrir em nova aba utilizando o atributo `target="_blank"`.
Após implementar, teste novamente no navegador.

Micro-Commit 3 — Links e Comentários

Registre a evolução do projeto:

```
git add .
git commit -m "Adicionando links externos e comentário HTML"
```

Parte 5 — Versionamento com Git e GitHub

Atividade 5 — Publicando o Projeto no GitHub

Passo 1 — Criar repositório no GitHub

- Acesse: `https://github.com`
- Clique em **New Repository**
- Nomeie como: `prog-internet-2026`
- Selecione como **Public**
- NÃO marque a opção para criar README automaticamente
- Clique em **Create repository**

Passo 2 — Executar comandos no terminal

No diretório do seu projeto, execute:

```
echo "# prog-internet-2026" >> README.md
git init
git add README.md
git add .
git commit -m "Primeiro commit - Estrutura inicial"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/prog-internet-2026.git
git push -u origin main
```

Se solicitado, utilize seu usuário e token de autenticação do GitHub.

Resultado Esperado

- O arquivo `index.html` deve estar visível no repositório;
- O arquivo `README.md` deve estar criado;
- O repositório deve conter pelo menos um commit inicial.

Boas Práticas a partir desta Aula

- Todo código desenvolvido será versionado;
- Cada etapa significativa deverá gerar um commit;
- O histórico de commits fará parte da avaliação formativa;
- Organização e clareza no repositório são critérios avaliativos.

Checklist Final de Verificação

Checklist Técnico

Antes de finalizar o roteiro, verifique:

- O documento inicia com `<!DOCTYPE html>`;
- A tag `<html>` contém o atributo `lang="pt-BR"`;
- O `<meta charset="UTF-8»` está corretamente definido;
- O código está indentado e organizado;
- Os links utilizam `target="_blank"`;
- O repositório contém pelo menos três commits distintos;
- O arquivo está corretamente publicado no GitHub.

Observação Técnica Importante

A tag `<font>` foi descontinuada no HTML5.
Toda formatação visual deve ser realizada utilizando CSS, que será introduzido nos próximos roteiros.

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Utilizar corretamente HTML5 semântico;
- Organizar múltiplas páginas interligadas;
- Criar e conectar um arquivo CSS externo;
- Trabalhar com imagens utilizando caminhos relativos;
- Estruturar diretórios de forma profissional;
- Aplicar versionamento incremental com Git.

Introdução

No roteiro anterior construímos uma página HTML básica e iniciamos o versionamento com Git.

Agora avançaremos para um modelo mais próximo de aplicações reais, organizando um mini-site com múltiplas páginas, estrutura semântica adequada e separação entre conteúdo (HTML) e estilo (CSS).

A separação entre estrutura e apresentação é um dos pilares do desenvolvimento web moderno. O HTML é responsável pela organização do conteúdo, enquanto o CSS define a aparência visual da aplicação.

Neste roteiro construiremos um mini-site institucional com organização profissional.

Fundamentação Teórica — HTML5 Semântico

O HTML5 introduziu uma abordagem mais semântica para organização de páginas web. Isso significa que as tags passaram a descrever o significado estrutural do conteúdo.

A semântica melhora:

- Organização do código;
- Acessibilidade;
- SEO (Search Engine Optimization);
- Manutenção e escalabilidade do projeto.

Principais Tags Estruturais do HTML5

Tag	Função Estrutural
<header>	Cabeçalho da página ou seção.
<nav>	Área de navegação principal.
<main>	Conteúdo principal da página.
<section>	Agrupamento temático de conteúdo.
<article>	Conteúdo independente (post, card, notícia).
<aside>	Conteúdo complementar ou lateral.
<footer>	Rodapé da página ou seção.
<link>	Conexão com arquivos externos (CSS).
<img>	Inserção de imagem (com atributo alt obrigatório).

Boas Práticas

Evite estruturar páginas apenas com . Utilize as tags semânticas para tornar o código mais claro e profissional.

Dicas Úteis — VS Code

Atalhos Importantes no VS Code para HTML

- Digite `!` e pressione `TAB` para gerar automaticamente a estrutura básica do HTML5.
- Para formatar automaticamente o código:
  - `OPTION + SHIFT + I` [Mac].
  - `ALT + SHIFT + I` [Linux].
  - `ALT + SHIFT + F` [Windows].
- Utilize `CTRL + /` para comentar ou descomentar linhas.
- Use `CTRL + Espaço` para ativar sugestões automáticas.
- Utilize `CTRL + S` frequentemente para salvar alterações.

Dica Profissional: Acostume-se a trabalhar com código bem indentado e organizado desde o início. Isso faz parte da avaliação formativa.

Dicas Úteis — VS Code

Sequência Recomendada de Uso do Git

Sempre que finalizar uma etapa do trabalho, utilize a seguinte sequência:

```
git status
git add .
git commit -m "Descrição clara da alteração"
git push
```

Boa prática:

- Faça commits pequenos e frequentes;
- Utilize mensagens descritivas;
- Evite acumular muitas alterações em um único commit.

Parte 1 — Organização Profissional do Projeto

Estrutura recomendada:

```
prog-internet-2026/

 roteiro02/
     index.html
     sobre.html
     contato.html
     assets/
         css/
            style.css
         img/
             perfil.jpg
```

Atividade 1 — Criando Estrutura Base

- Criar pasta `roteiro02`;
- Criar os três arquivos HTML;
- Criar pasta `assets/css`;
- Criar pasta `assets/img`;
- Criar arquivo `style.css`.

Micro-Commit 1

```
git add .
git commit -m "Criando estrutura profissional roteiro 02"
```

Parte 2 — Estrutura HTML Semântica

No arquivo `index.html`:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
```

```
    <title>Mini-Site</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>

<header>
    <h1>Meu Mini-Site</h1>
</header>

<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="sobre.html">Sobre</a></li>
        <li><a href="contato.html">Contato</a></li>
    </ul>
</nav>

<main>
    <section>
        <h2>Conteúdo Principal</h2>
        <p>Texto da seção.</p>
    </section>
</main>

<footer>
    <p>Seu nome - 2026</p>
</footer>

</body>
</html>
```

Atividade 2 — Implementando Estrutura

- Implementar estrutura nas três páginas;
- Garantir navegação funcional;
- Testar todos os links.

Micro-Commit 2

```
git add .
git commit -m "Implementando estrutura semantica completa"
```

Parte 3 — Introdução ao CSS Externo

No arquivo `style.css`:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
}

header {
    background-color: #003366;
    color: white;
    padding: 15px;
}

nav ul {
    list-style: none;
}

nav li {
    display: inline;
    margin-right: 15px;
}

footer {
    margin-top: 20px;
    font-size: 14px;
}
```

Atividade 3 — Aplicando CSS

- Criar e conectar arquivo CSS;
- Verificar funcionamento nas três páginas;
- Ajustar estilos conforme necessário.

Micro-Commit 3

```
git add .
git commit -m "Adicionando estilo CSS externo"
```

Parte 4 — Inserindo Imagens

No arquivo `index.html`:

```
<img src="assets/img/perfil.jpg" alt="Foto de perfil">
```

Atividade 4 — Trabalhando com Imagem

- Inserir imagem na página inicial;
- Utilizar atributo alt;
- Garantir caminho relativo correto.

Micro-Commit 4

```
git add .
git commit -m "Adicionando imagem ao mini-site"
```

Checklist Final

Verificação Estrutural

- Estrutura de diretórios organizada;
- HTML semântico implementado corretamente;
- CSS externo funcionando;
- Navegação funcional;
- Imagem carregando corretamente;
- Pelo menos 4 commits realizados.

Publicação Final no GitHub

Após revisar o checklist:

- Verifique se todos os arquivos estão salvos;
- Execute a sequência completa de versionamento;
- Certifique-se de que o repositório remoto está atualizado.

```
git status
git add .
git commit -m "Finalizacao roteiro 02"
git push
```

Resultado esperado:
Todos os arquivos do roteiro02 devem estar visíveis no GitHub.

Entrega da atividade

Entrega da Atividade

Para concluir esta atividade, o estudante deverá:

- Confirmar que o repositório está atualizado no GitHub;
- Copiar o link público do repositório;
- Enviar apenas o link na tarefa correspondente no SIGAA.

Exemplo de link esperado:
`https://github.com/seu-usuario/prog-internet-2026`
Importante:

- O repositório deve estar público;
- O histórico de commits será considerado na avaliação;
- Não será necessário enviar arquivos compactados (.zip).

Fundamentação Complementar — Propriedades CSS Essenciais

O CSS (Cascading Style Sheets) é responsável pela apresentação visual da página. Ele permite definir cores, espaçamentos, fontes, posicionamento e organização visual.

Principais Propriedades CSS

Propriedade	Função
color	Define a cor do texto.
background-color	Define a cor de fundo do elemento.
font-family	Define a família da fonte.
font-size	Define o tamanho da fonte.
margin	Define o espaçamento externo do elemento.
padding	Define o espaçamento interno do elemento.
border	Define bordas ao redor do elemento.
width / height	Define largura e altura do elemento.
display	Define o tipo de exibição (block, inline, inline-block).
text-align	Define alinhamento do texto.

Exemplo Prático

```
section {
    background-color: #f4f4f4;
    padding: 20px;
    margin: 10px;
}
```

Dicas Essenciais — Git e GitHub

O Git é a ferramenta de controle de versão utilizada nesta disciplina. Abaixo estão os principais comandos que você utilizará com frequência.

Comando	Função
git status	Mostra o estado atual do repositório (arquivos modificados, adicionados ou pendentes de commit).
git add .	Adiciona todos os arquivos modificados para a área de preparação (staging area).
git commit -m "mensagem"	Cria um novo commit com uma descrição clara da alteração realizada.
git push	Envia os commits locais para o repositório remoto no GitHub.
git pull	Atualiza seu repositório local com alterações feitas no repositório remoto.

4 Roteiro Prático 03 — Publicação com GitHub Pages e Aprimoramento Visual

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Refinar a estrutura visual do mini-site;
- Organizar o layout utilizando container centralizado;
- Aplicar efeitos visuais simples com CSS;
- Configurar e publicar o site utilizando GitHub Pages;
- Disponibilizar um link público funcional.

Introdução

Até este momento construímos um mini-site estruturado com HTML5 semântico e CSS externo.

Agora daremos um passo importante: tornar o projeto visualmente mais organizado e publicá-lo na internet utilizando o GitHub Pages.

A publicação de um site estático permite que qualquer pessoa acesse sua aplicação por meio de um endereço público.

Este é o primeiro contato com deploy web na disciplina.

Parte 1 — Ajustes Estruturais

Adicionar dentro do <head> de todas as páginas:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Isso prepara o site para futura responsividade.

Parte 2 — Aprimoramento Visual com CSS

Atualize o arquivo `style.css` com:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background-color: #f2f2f2;
}

.container {
    width: 80%;
    margin: 0 auto;
}

header {
    background-color: #003366;
    color: white;
    padding: 20px;
    text-align: center;
}

nav ul {
    list-style: none;
    padding: 0;
    text-align: center;
}

nav li {
    display: inline;
    margin: 0 15px;
}

nav a {
    text-decoration: none;
    color: #003366;
    font-weight: bold;
}

nav a:hover {
    color: #ff6600;
```

```
}

section {
    background-color: white;
    padding: 20px;
    margin-top: 20px;
    border-radius: 8px;
    box-shadow: 0px 2px 6px rgba(0,0,0,0.1);
}

footer {
    margin-top: 30px;
    padding: 15px;
    text-align: center;
    font-size: 14px;
}
```

Atividade 1 — Aplicando Container

- Envolver o conteúdo principal dentro de uma <div class="container»;
- Testar o alinhamento centralizado;
- Verificar se todas as páginas mantêm consistência visual.

Micro-Commit 1

```
git add .
git commit -m "Aprimorando layout com container e efeitos visuais"
```

Parte 3 — Publicação com GitHub Pages

4.3.1 Passo 1 — Atualizar repositório

```
git status
git add .
git commit -m "Preparando publicacao no GitHub Pages"
git push
```

4.3.2 Passo 2 — Ativar GitHub Pages

No repositório no GitHub:

- Acessar **Settings**;
- Clicar em **Pages**;
- Em Source, selecionar:
  - Branch: main
  - Folder: /root
- Salvar.

O GitHub fornecerá um endereço público no formato:
`https://seu-usuario.github.io/prog-internet-2026/`

4.3.3 Parte 4 — Verificação da Publicação

Atividade 2 — Teste de Acesso Público

- Acessar o link fornecido pelo GitHub;
- Navegar entre as páginas;
- Verificar funcionamento da navegação;
- Testar em outro navegador ou dispositivo.

5 Checklist Final

Verificação Geral

- Layout centralizado;
- CSS aplicado corretamente;
- Efeito hover funcionando;
- Navegação funcional;
- GitHub Pages ativo;
- Site acessível por URL pública.

Entrega da Atividade

O estudante deverá enviar no SIGAA:

- Link público do GitHub Pages;
- Link do repositório no GitHub.

Não é necessário enviar arquivos compactados.

6 Roteiro Prático 04 — Evoluindo o Layout com Flexbox

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Recuperar corretamente um projeto existente com Git;
- Evoluir um site já publicado;
- Aplicar Flexbox para organização de layout;
- Criar seções com múltiplos cards;
- Implementar responsividade básica com Media Query.

Introdução

Nos roteiros anteriores desenvolvemos um mini-site estruturado, versionado com Git e publicado no GitHub Pages.

Neste roteiro, daremos continuidade ao mesmo projeto, aplicando técnicas modernas de layout com Flexbox para melhorar a organização visual e aproximar o desenvolvimento de práticas profissionais.

Não criaremos um projeto novo. Evoluiremos o que já foi construído.

Preparação do Ambiente — Recuperando o Projeto

6.3.1 Passo 1 — Clonar o Repositório Individual

No terminal, execute:

```
git clone https://github.com/SEU-USUARIO/prog-internet-2026.git
```

6.3.2 Passo 2 — Acessar o Diretório

```
cd prog-internet-2026
```

6.3.3 Passo 3 — Entrar na Pasta do Último Roteiro

```
cd roteiro-3
```

Dicas Úteis — VS Code

Importante:
Nunca recrie manualmente um projeto que já existe no GitHub. Sempre utilize `git clone` para manter o histórico completo.

Fundamentação Teórica — Flexbox

Flexbox (Flexible Box Layout) é um modelo moderno de organização de layout unidimensional.

Ele permite alinhar e distribuir elementos em linha ou coluna de forma dinâmica.

6.4.1 Ativando o Flexbox

```
display: flex;
```

6.4.2 Principais Propriedades

Propriedade	Função
display: flex	Ativa o modo flexível.
flex-direction	Define direção (row ou column).
justify-content	Alinha no eixo principal.
align-items	Alinha no eixo secundário.
gap	Define espaçamento entre elementos.
flex-wrap	Permite quebra automática.

Visualização do Resultado Esperado

Abaixo está uma representação ilustrativa do layout final esperado após a aplicação do Flexbox e da responsividade.

Dicas Úteis — VS Code

Observação:
O layout pode variar visualmente (cores, textos e imagens), mas a organização estrutural deve seguir o modelo apresentado.

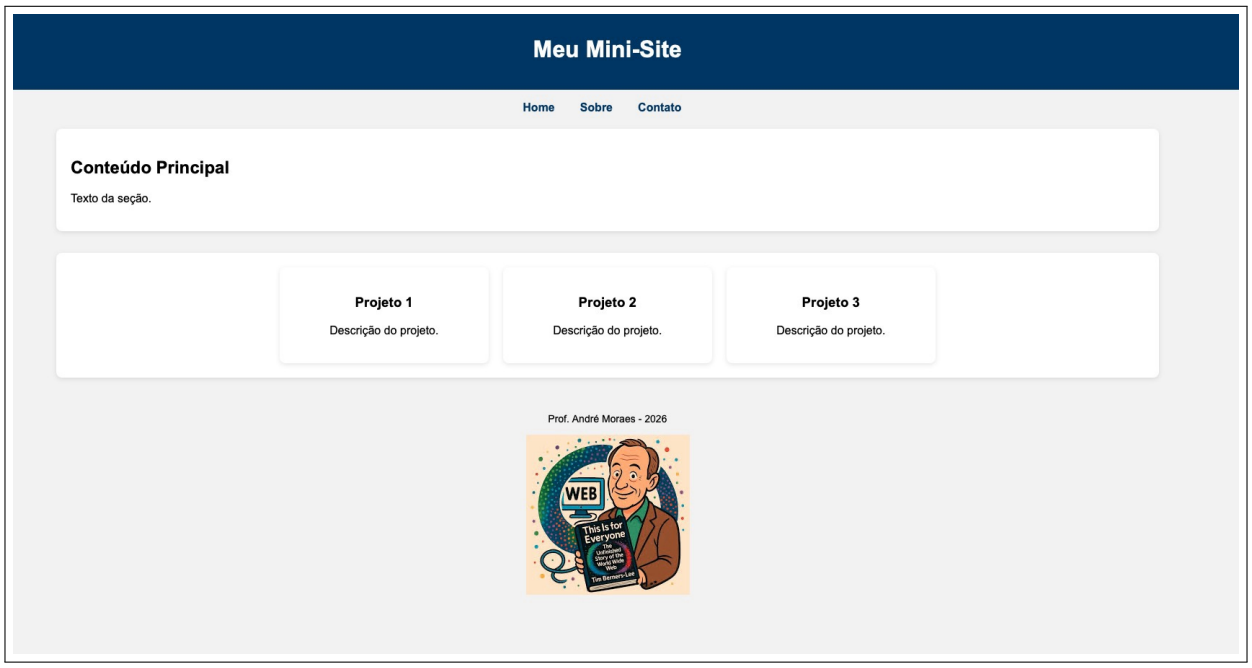

Figura 1: Exemplo de layout final com menu flexível e seção de cards.

Parte 1 — Melhorando o Menu com Flexbox

Atualize o CSS existente:

```
nav ul {
    display: flex;
    justify-content: center;
    gap: 20px;
    list-style: none;
    padding: 0;
}
```

Atividade 1 — Menu Flexível

- Aplicar Flexbox ao menu existente;
- Centralizar os itens;
- Ajustar espaçamento.

Micro-Commit 1

```
git add .
git commit -m "Aplicando Flexbox no menu"
git push
```

Parte 2 — Organizando Conteúdo com Cards

Adicione abaixo do conteúdo principal:

6.7.1 Estrutura HTML

```
<section class="cards">
    <div class="card">
        <h3>Projeto 1</h3>
        <p>Descrição do projeto.</p>
    </div>
    <div class="card">
        <h3>Projeto 2</h3>
        <p>Descrição do projeto.</p>
    </div>
    <div class="card">
        <h3>Projeto 3</h3>
        <p>Descrição do projeto.</p>
    </div>
</section>
```

6.7.2 Estilização CSS

```
.cards {
    display: flex;
    gap: 20px;
    justify-content: center;
    flex-wrap: wrap;
    margin-top: 30px;
}

.card {
```

```
    background-color: white;
    padding: 20px;
    width: 250px;
    border-radius: 8px;
    box-shadow: 0px 2px 8px rgba(0,0,0,0.1);
    text-align: center;
}
```

Atividade 2 — Implementando Cards

- Criar três cards reais;
- Ajustar visual;
- Testar quebra automática.

Micro-Commit 2

```
git add .
git commit -m "Adicionando layout com cards em Flexbox"
git push
```

Parte 3 — Responsividade Básica

Adicione ao final do CSS:

```
@media (max-width: 768px) {
    .cards {
        flex-direction: column;
        align-items: center;
    }
}
```

Atividade 3 — Teste de Responsividade

- Redimensionar o navegador;
- Verificar reorganização dos cards;
- Testar em dispositivo móvel.

Micro-Commit 3

```
git add .
git commit -m "Adicionando responsividade com media query"
git push
```

Checklist Final

Verificação Geral

- Projeto clonado corretamente;
- Menu utilizando Flexbox;
- Seção de cards implementada;
- Responsividade funcionando;
- Commits organizados;
- GitHub Pages atualizado.

Repositório Oficial da Unidade Curricular

Repositório Oficial da Disciplina

`https://github.com/chameoandre/uc-prog-internet-2026`

Todos os roteiros, exemplos e materiais complementares são disponibilizados neste endereço. Recomenda-se acompanhar atualizações ao longo do semestre.

6. FAQ – Problemas Comuns com Git e GitHub

Durante as atividades práticas com Git e GitHub, alguns erros podem ocorrer, principalmente em ambientes de laboratório Linux. Abaixo estão os problemas mais comuns e como resolvê-los rapidamente.

6.1 Erro: Please tell me who you are

Mensagem típica:

```
*** Please tell me who you are.
Run
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"
```

Causa: O Git precisa identificar quem está realizando o commit. Nome e e-mail ainda não foram configurados na máquina.

Solução:

Execute no terminal:

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu_email@exemplo.com"
```

Exemplo:

```
git config --global user.name "Maria da Silva"
git config --global user.email "maria@email.com"
```

Após isso, tente novamente:

```
git commit -m "Mensagem do commit"
```

Observação: Se o computador for compartilhado, pode-se configurar apenas para o repositório atual:

```
git config user.name "Seu Nome"
git config user.email "seu_email@exemplo.com"
```

—

6.2 Erro: "fatal: unable to auto-detect email address

Causa: Mesmo problema do item anterior: o Git não tem nome e e-mail configurados.

Solução: Configurar o usuário conforme indicado na seção 6.1.

—

6.3 Erro ao dar push – senha não aceita

Mensagem típica:

```
remote: Support for password authentication was removed
fatal: Authentication failed
```

Causa: O GitHub não permite mais autenticação usando apenas senha.
Soluções possíveis:
Opção 1 – Utilizar Token de Acesso (PAT):

- Gerar um token no GitHub.
- Usar o token no lugar da senha ao executar o `git push`.

Opção 2 – Configurar autenticação via SSH:

- Gerar chave SSH com:

  ```
  ssh-keygen -t ed25519 -C "seu_email@exemplo.com"
  ```

- Adicionar a chave pública nas configurações do GitHub.

—

6.4 Verificar configurações atuais do Git

Para conferir se o nome e e-mail estão configurados:

```
git config --list
```

Ou apenas as configurações globais:

```
git config --global --list
```

—

6.5 Erro: "nothing to commit

Mensagem típica:

```
nothing to commit, working tree clean
```

Causa: Nenhuma alteração foi adicionada à área de stage.

Solução:

Verificar o status:

```
git status
```

Adicionar arquivos:

```
git add nome_do_arquivo
```

Ou todos os arquivos:

```
git add .
```

Depois realizar o commit novamente.

—

6.6 Dica Importante

Sempre que ocorrer um erro:

- Leia atentamente a mensagem exibida no terminal.
- Utilize o comando `git status` para entender a situação atual do repositório.
- Evite repetir comandos sem compreender o erro.

Grande parte dos problemas com Git está relacionada à configuração inicial ou à autenticação.